SQL Injection Attacks
Object Relnl Models
Recursive SQL queries

Making SQL queries by sticking strings together is <u>awful</u>

- you open yourself up to SQL injection attacks

```
import psycopg2

conn = psycopg2.connect("user=dmusicant dbname=dmusicant")

cur = conn.cursor()

name = input('Which recipe? ')

query = "SELECT * FROM recipes WHERE name= %(name)s"


print(query)
cur.execute(query, {"name":name})
for row in cur:
    print(row)
~
~
~
```

Object Rehl Models are toolkits
that wrap up reln models in
object-oriented programming.
- Django
- SQL Alchemy

~~Transform~~ an underlying DB
Takes
and gives you a wrapper
for using objects/methods
to interact w/ it

Pros - Python!! (or whatever)

Cons - complex queries are
a mess - sometimes
endup going to SQL

Recursive queries
(Wow, SQL can do that?)

```
(5 rows)

dmusicant=> select * from immediate_prereqs;
 prereq_id | course_id
-----------+-----------
 CS111     | CS201
 CS201     | CS257
 CS257     | CS347
 CS201     | CS251
(4 rows)
```

What I want is a table that find all courses each course is necessary for, even if a chain of courses is necessary to get there

prereq_id	course_id
cs111	cs201
cs111	cs257
cs111	cs347
cs201	cs257
cs201	cs347

. . . . .

temp table

```
-- thing
    WITH RECURSIVE all_reqs (req_id, course_id) AS (
            SELECT prereq_id,
                   course_id
              FROM immediate_prereqs
             UNION
            SELECT AR.req_id,
                   IP.course_id
              FROM all_reqs AR
              JOIN immediate_prereqs IP
                ON AR.course_id = IP.prereq_id
        )
  SELECT *
    FROM all_reqs;
~
```

base case "anchor"

use the temp table

Think about it:

1. The base case runs, all tuples are added to the result, and also a temp working table
2. Do the query after UNION, using the working table

```
dmusicant=> select * from immediate_prereqs;
 prereq_id | course_id
-----------+-----------
 CS111     | CS201
 CS201     | CS257
 CS257     | CS347
 CS201     | CS251
(4 rows)
```

① all_reqs

req_id	course_id
111	201
201	257
257	347
201	251

working table

req_id	course_id
111	201
201	257
257	347
201	251

② do the join

req_id	course_id
111	257
111	251
201	347

(3) new values set added to result, and replace working table

req-id	course-id
111	201
201	257
257	347
201	251
111	257
111	251
201	347

working table	
111	257
111	251
201	347

Repeat again

111	347